

Message-Driven Architecture

4TH YEAR - ENSIA

DISCUSSION

For each scenario, decide the right communication style and justify your choice in one sentence. There is no single correct answer for every row – defend your reasoning.

Scenario	REST or Kafka?	Key reason
User clicks “Login” – credentials must be validated immediately and the user redirected.		
When a document is uploaded, it must be indexed for search, virus-scanned, and translated into 3 languages.		
An audit log of every read access must be persisted, but it must not slow down the read path.		
10 downstream services need the same “document uploaded” notification.		

INDIVIDUAL EXERCISES:

Task / Exercise C1 [20 Minutes] (Hello World Kafka)

Get a single-broker Kafka cluster running on your machine. Use Docker for the broker (Bitnami or Confluent images both work) and add a small web UI (Kafka UI or Redpanda Console) so you can inspect topics and messages from your browser.

When you are done, the following must be true:

- The broker is running and reachable on a known port.
- The web UI is open in your browser and shows the broker as healthy.
- You have created a test topic and used the UI

Task / Exercise C2 [30 Minutes] (Documents Service as Producer)

Create the topic `dms.documents.uploaded` with 3 partitions, then modify the Documents service so that every successful upload publishes a `DocumentUploaded` event to it. The event must carry enough information for a downstream service to act on it without calling back.

Design decisions you must make and document:

1. What goes inside the event payload? What is deliberately left out, and why?
2. What field do you choose as the message key?

Task / Exercise C3 [40 Minutes] (Python Translator Service)

Build a small Python microservice that consumes the `translation-request` events from Kafka, calls an AI-API service to produce the translation, and publishes the result to a second Kafka topic. AI-API service include one of the following:

- OpenRouter (use the free models).
- Gemini
- Groq
- <https://cloud.cerebras.ai/>
- ..

Architectural rule for this exercise: the translator must not call the Documents service directly. It only talks to Kafka – it reads from one topic, calls the AI service, and writes to another topic. Persisting the translation back into the DMS is a separate concern that belongs to a different service . *Be ready to defend this decision: why is calling DMS directly a regression even though it would “work”?*

When you are done:

- The service is containerized and joins a consumer group named translator-service.
- It reads from dms.documents.uploaded, calls Gemini, and publishes the translated text to dms.documents.translated.
- Sending one document through the Gateway in Postman results in two messages visible in the Kafka UI: the original request, and a few seconds later, the translated result on the second topic.

Decisions you must justify:

1. Auto-commit offsets, or commit manually? When exactly should the offset be committed relative to the Gemini call and the publish?
2. What auto.offset.reset policy makes sense for this service – earliest or latest?
3. Where is the Gemini API key stored? Why never inside the image?
4. What happens if Gemini is unreachable? Retry, skip, or fail loudly?

PROGRESSING ON THE NEXT DELIVERABLE DUE 02 MAY 2026:

Students should continue to work on their next increment which includes:

- Partial Integration of Front-end with the Backend
- Simple creation of documents from the Front-end and storing them into the backend.
- Commenting system within the document.
- Use of
 - Redis, Cassandra, Postgres for data storage of different types of records.
 - Use of S3 for storing binary files + use of pre-signed URLs.
- Use of Event-driven architecture via Kafka for :
 - Translation Services with Gemini API + Python Service
 - Translation needs to be written to the Postgres database of the DMS.
 - Either using the same document or you create a new micro service to consume the translated messages.
- Deployment within K8

FINAL DELIVERABLE DUE 16 MAY 2026:

- Fully working DMS.
- Architecture Report.
 - Architecture Diagram
 - Defend why it is enterprise dealing with :
 - Scalability
 - Recovery from disasters/High SLA
 - Availability
 - Consistency
 - ..